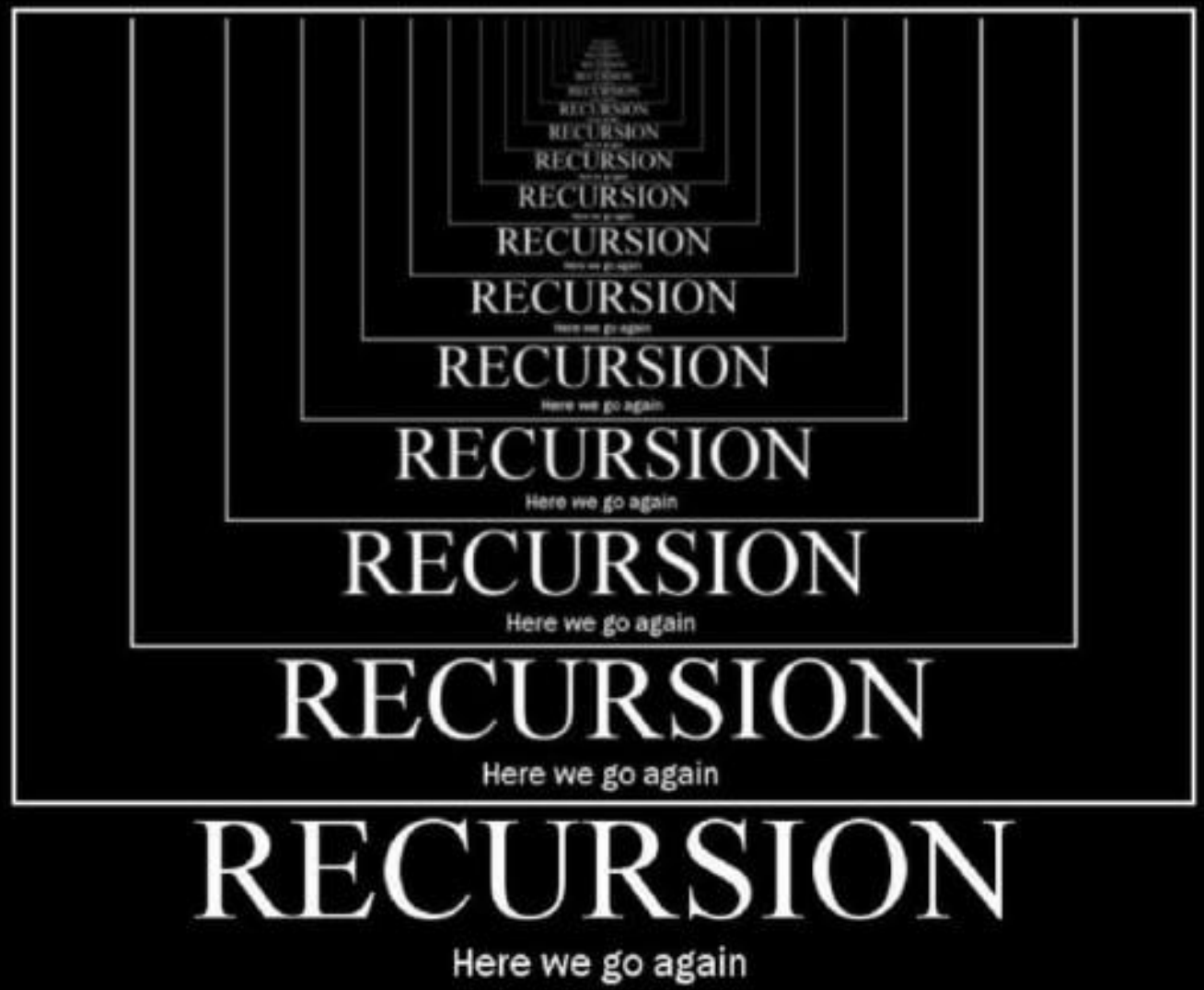




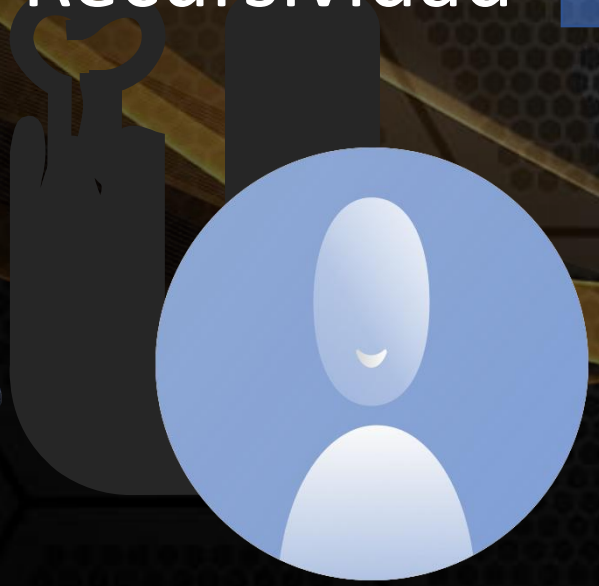
U6 - Recursión

recursion


RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again
RECURSION
Here we go again



U5 - Recursividad





Recursión | Introducción

CORE

Recursión | Ejemplo Factorial
Recursión | recurence

Recursión | uso de la pila en la recursión
Recursión | Ordenar una pila mediante recursión – Example

TIPOS

Recursión | Variants of Recursion
Recursión | Variants of Recursion | Direct recursion
Recursión | Variants of Recursion | tail recursion
Recursión | Variants of Recursion | Linear Recursion
Recursión | Variants of Recursion | Tree Recursion

EJEMPLOS

Recursión | Execution of recursive calls
Recursión | Writing Recursive Code

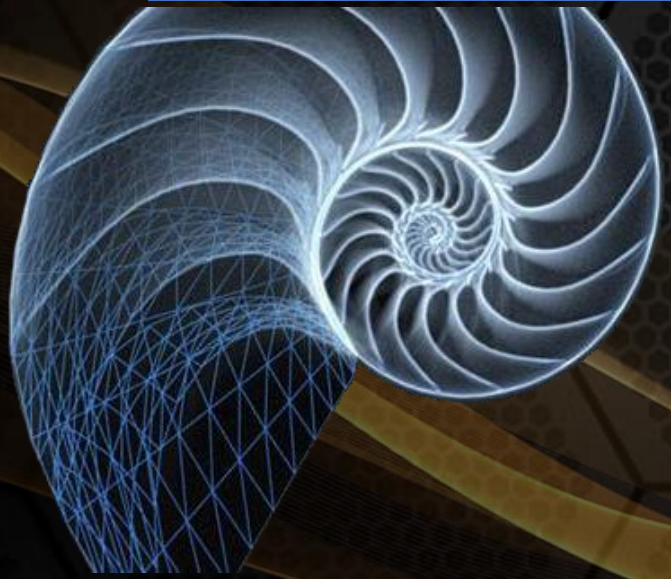
Recursión | Torre de Hanoi
Recursión | Writing Recursive Code – Exmaple

CONCEPTOS

Recursión | Comprobación de la corrección
Recursión | Iteración versus Recursión
Recursión | Simulación de recursión mediante stack
Recursión | Aplicaciones - Resumen
Recursión | Resumen

Recursión| C++
Introducción.





La recursión o recursividad es la forma en la cual se especifica un proceso basado en su propia definición. La recursión tiene esta característica discernible en términos de autorreferencialidad, autopoiesis, fractalidad o, en otras palabras, construcción a partir de un mismo tipo. Con ánimo de una mayor precisión, y para evitar la aparente circularidad en esta definición, se formula el concepto de recursión de la siguiente manera:

Un problema que pueda definirse en función de su tamaño, sea este N, puede dividirse en instancias más pequeñas (< N) del mismo problema y se conocerá la solución explícita a las instancias más simples, lo que se conoce como casos base, y se puede aplicar inducción sobre las llamadas más pequeñas y suponer que estas quedan resueltas al juntarlas.



Recursión | Recursion

Recursión

Introducción.

Las funciones son la característica más básica y útil de cualquier lenguaje de programación. Un conjunto de instrucciones que realiza operaciones lógicas, que pueden ser muy complejas y numerosos en número, se pueden agrupar como funciones (también llamados procedimientos).
Las funciones pueden llamarse a sí mismas o a otras funciones, y las funciones llamadas a su vez pueden llamar a la función de llamada. Este proceso se llama recursión y tales funciones se llaman
funciones recursivas. Una función recursiva hace que el programa sea compacto y legible.

Recursión | Introducción

Las buenas prácticas de programación enfatizan la escritura de programas que son legibles, fáciles de entender y libres de errores. Las funciones son la característica más útil que logra esto. Se llama a una función utilizando un nombre de función y sus parámetros a través de instrucciones. Dada la especificación de entrada-salida de una función, la persona que llama simplemente le hace una llamada. *Esta vista de las funciones implica que se invoca, ejecuta y devuelve (con o sin resultados) al lugar donde se llamó en la función de llamada.* Cuando una función se llama a sí misma, ya sea directa o indirectamente, se dice que está haciendo una llamada recursiva.

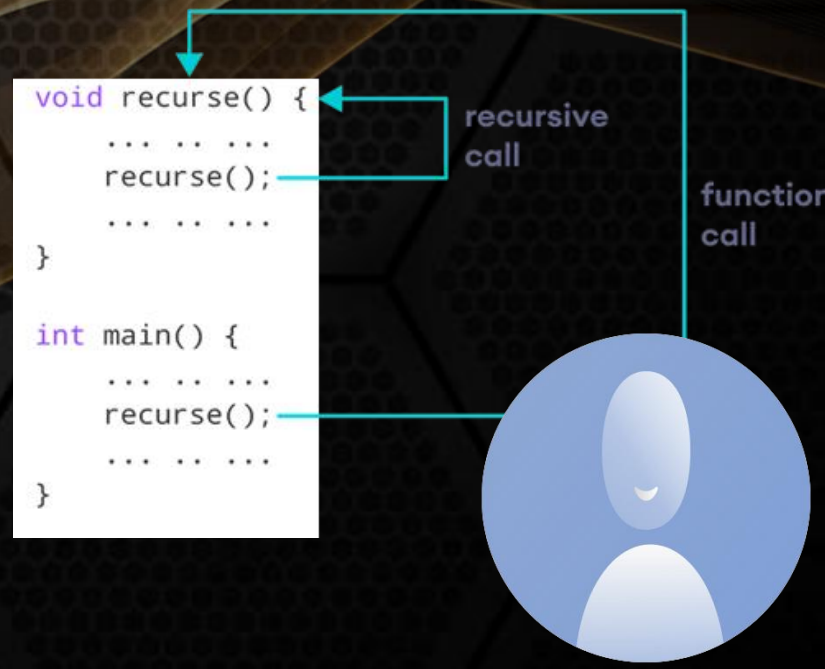
Un programa se vuelve compacto y legible con funciones recursivas. La recursión es extremadamente poderosa, ya que permite al programador expresar procesos complejos fácilmente. Los programas recursivos se utilizan en una variedad de aplicaciones que van desde calcular el factorial de un número hasta jugar juegos complejos contra la inteligencia humana.

Consideremos un ejemplo de calcular el factorial de un número.
Factorial es un término matemático. El factorial de un número, digamos n, es igual al producto de todos los enteros de 1 a n. El factorial de n se denota como

n! = 1 x 2 x 3 x ... x n or n! = n x n - 1 x ... x 1

Por ejemplo, ¡10! = 1 x 2 x 3 x 4 x 5 x 6 x 7 x 8 x 9 x 10.

El programa más simple para calcular el factorial de un número es mediante el uso de un bucle con una variable de producto. ¡El algoritmo establece el proceso iterativo de calcular el factorial de n como 10! = 10 x 9 x 8 x ... x 1



Recursión | Introducción

Los algoritmos 4.1 y 4.2 son **algoritmos iterativos** para calcular el factorial de n. También es posible dar una definición recursiva para factorial. La función matemática definida en Eq. (4.1) para factorial de n también puede ser desafiada recursivamente como n! = n x (n - 1)!, donde 1! = 1 Esta definición recursiva de factorial tiene dos pasos, de la siguiente manera:

1. Si n = 1, entonces factorial de n = 1
2. De lo contrario, factorial de n = n x factorial de (n - 1)

```
int Factorial(int n)
{
 if(n == 1) // end condition
 return 1;
 else
 return Factorial(n - 1) * n;
}
```

```
int main() {
    int n, result;

    cout << "Enter a non-negative number: ";
    cin >> n;

    result = factorial(n);
    cout << "Factorial of " << n << " = " <<
result;
    return 0;
}
```

La función Factorial() es un ejemplo de una función recursiva. En la segunda instrucción return, la función llama asi misma. Lo importante a recordar al crear una función recursiva es dar una condición final. En el código de programa 4 .1, la recursión se detiene cuando n se convierte en 1. En cada llamada de la función, el valor de n sigue disminuyendo. Sin embargo, cuando el valor alcanza 1, la función termina. Por otro lado, esta función se ejecutará infinitamnete si el valor inicial de n es menor que 1, lo que significa que la función no es perfecta. Por lo tanto, la condición n = 1 debe cambiarse a n ≤ 1. L reescriben la función Factorial() como en el Código de Programa 4.2

ALGORITHM 4.1

```
An iterative version of an algorithm to compute the factorial of a
number
1. start
2. Let n be the number whose factorial is to be computed and let
   Factorial = 1
3. while(n > 1) do
   begin
     Factorial = Factorial * n
     n = n - 1
   end
4. stop
```



ALGORITHM 4.2

```
An iterative version of the algorithm to compute the factorial of a
number
1. start
2. Let n be the number whose factorial is to be computed and let
   Factorial = 1
3. for I = 1 to n do            // I can also be initialized to 2
   begin
      Factorial = Factorial * I
   end
4. stop
```

El Código de Programa 4.2 aprovecha el hecho de que el factorial de cualquier entero n puede definirse recursivamente como el producto de n y el factorial de n − 1. Por ejemplo, ¡5! = 5 x 4

Recursión | Introducción

Algoritmo 5.1 es una Versión iterativa de un algoritmo para calcular el factorial de un número. 2. Sea n el número cuyo factorial se va a calcular y sea Factorial = 1

```
int Factorial;
While (n > 1) {
Factorial = Factorial * n ;
n = n - 1 ;
}
```

Programa 5.1

El proceso iterativo de calcular el factorial de n en el Algoritmo 5.1 también se puede escribir como en el Algoritmo 5.2. Una versión iterativa del algoritmo para calcular el factorial de un número, Sea n el número cuyo factorial se va a calcular y sea Factorial = 1

```
// I can also be initialized to 2
for (I = 1 , I <= n , I++)
 {
 Factorial = Factorial * I
}
```

Programa 5.2

Los algoritmos 4.1 y 4.2 son algoritmos iterativos para calcular el factorial de n. Es posible dar una definición recursiva para factorial también.

La función matemática definida en Eq. (4.1) para factorial de n también se puede definir recursivamente como:

n! = n * (n - 1)!, donde 1! = 1 (4.2)

Esta definición recursiva de factorial tiene dos pasos, de la siguiente manera:

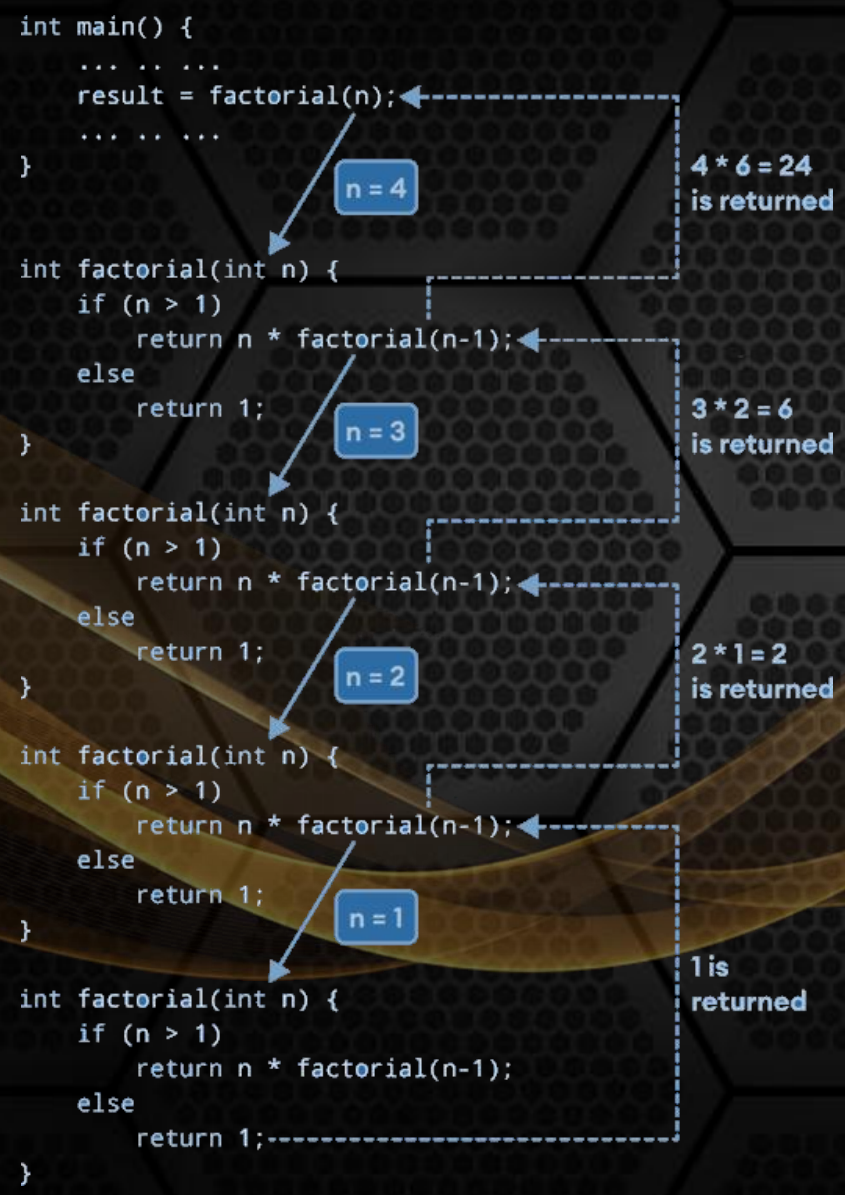
1. Si n = 1, entonces factorial de n = 1
2. De lo contrario, factorial de n = n * factorial de (n - 1)

```
//Program Code 5.2
int Factorial(int n)
{
 if(n == 1) // end condition
 return 1;
 els e
 return Factorial(n - 1) * n;
}
```



Recursión | Ejemplo Factorial

Como podemos ver, la función factorial() se llama a sí misma. Sin embargo, durante cada llamada, hemos disminuido el valor de n en 1. Cuando n es menor que 1, la función factorial() finalmente devuelve la salida.




```cpp
// Factorial of n = 1*2*3*...*n

#include <iostream>
using namespace std;

int factorial(int);

int main() {
    int n, result;

    cout << "Enter a non-negative number: ";
    cin >> n;

    result = factorial(n);
    cout << "Factorial of " << n << " = " << result;
    return 0;
}


int factorial(int n) {
    if (n > 1) {
        return n * factorial(n - 1);
    } else {
        return 1;
    }
}
```



Factorial.cpp

Recursión | Recurrencia Recurence

Una recurrencia es una función matemática bien definida donde la función que se está definiendo se aplica dentro de su propia definición. El factorial lo definimos como n! = n x (n - 1)! es un ejemplo de recurrencia con 1! = 1 como condición final.

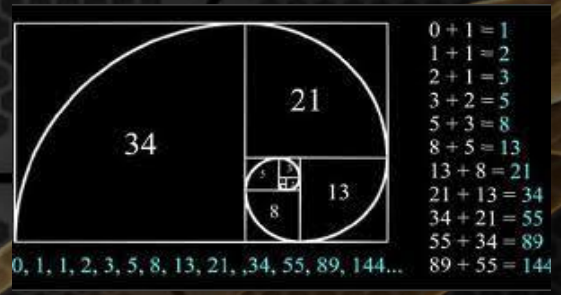
Ahora Tomemos como ejemplo la secuencia de Fibonacci. La secuencia de Fibonacci es la secuencia de los números
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...
Los dos primeros números de la secuencia son ambos 1, mientras que cada número siguiente es la suma de los dos números anteriores (llegamos al 55 el decimo; y es la suma del 21 y 34, los números octavo y noveno).
Definamos una función F(n) que devuelve el (n + 1) ésimo número de Fibonacci.
Primero, definimos los casos base (esto se conoce como condición de diseño) representados por las siguientes funciones:

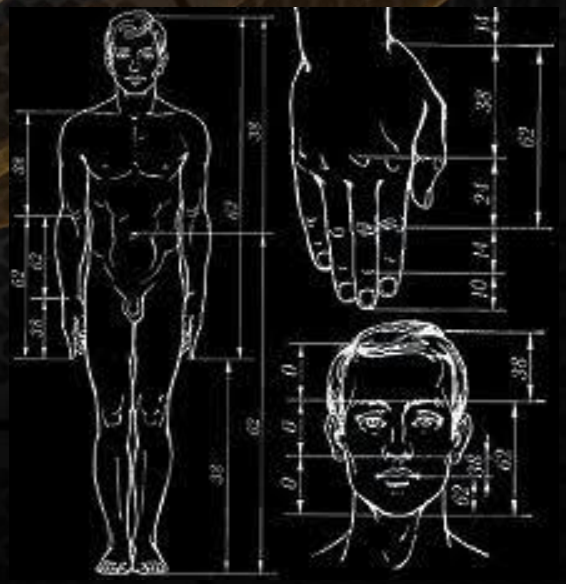
F(1) = 1 y F(2) = 1

Ahora, consideramos los otros números. Para obtener el (n + 1) ésimo número de Fibonacci, simplemente sumamos el enésimo y el (n - 1) ésimo número de Fibonacci.

F(n) = F(n - 1) + F(n – 2)

Esta función F se denomina **recurrencia** ya que calcula el enésimo valor en términos de (n - 1)th y (n - 2) th valores de Fibonacci. Los problemas que se pueden describir usando recurrencia se expresan fácilmente como funciones recursivas en la programación. El proceso de recursión ocurre cuando una función se llama a sí misma.













Recursión | recurence



El proceso de recursión ocurre cuando una función se llama a sí misma. La recursión es útil en situaciones en las que resolver una o más versiones más pequeñas del mismo problema puede resolver el problema. Calculando el valor de tres a la cuarta potencia se puede considerar como $3^4 = 3 \times 3^3$ Tres cubos se pueden definir como $3^3 = 3 \times 3^2$ Tres al cuadrado es $3^2 = 3 \times 3$ finalmente, $3 = 3 \times 3^0 = 3 \times 1$ La recurrencia para este cálculo es $X^Y = X \times X^{Y-1}$

```
Long int Power(int x, int y)
{
 if(y == 0) // end condition
 return(1);
 else
 return( x * Power(x, y – 1);
 // This is the “recursive call”
}
```

En cada uno de estos casos, el problema se reduce a una versión más pequeña de sí mismo. El código de programa 4.3 es un código recursivo para el cálculo de Eq. (4.4).

```
Long int Power(int x, int y)
{
 int result = 1;
 for(int i = 1; i <= y; i++)
 result = result * x;
 return(result);
}
```

La condición final en el código de programa 4.3 puede ser y = 1; entonces el valor devuelto será x (Código de programa 4.6). La versión iterativa del mismo cálculo se muestra en el código de programa 4.4.

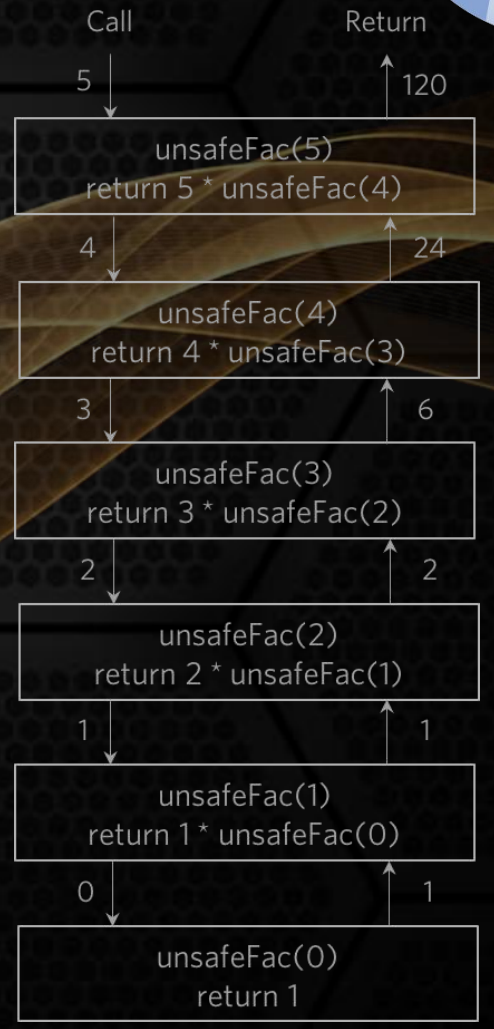
Recursión | uso de la pila en la recursión



Hemos estudiado la pila como estructura de datos en la Unidad 5. La pila en S.O es un área especial de memoria donde se almacenan variables temporales. Actúa sobre la base del principio LIFO. Para entender cómo las funciones recursivas usan la pila, analicemos el Código de Programa 4.2. Los pasos principales se dan en el siguiente código:

```
int factorial(int n) {

    if (int I <= 1){
        return 1;
    }
    else{
        return n * Factorial(n - 1);
    }
}
```



Sea n = 3; es decir, calculemos el valor de 3!, que es 3 x 2 x 1 = 6. Cuando la función se llama f o el primer tiempo, n contiene el valor 3, por lo que se ejecuta la instrucción else. La función conoce el valor de n pero no de Factorial(n − 1) por lo que empuja n (valor = 3) dentro de la pila y se llama a sí mismo por segunda vez con el valor 2. Esta vez, la instrucción else se ejecuta de nuevo, y n (valor = 2) se empuja a la pila a medida que la función se llama a sí misma por tercera vez con el valor 1. Ahora, la instrucción if se ejecuta y como n = 1, la función devuelve 1 . Como ahora se conoce el valor de Factorial(1), se revierte a su segunda ejecución extrayendo el último valor 2 de la pila y multiplicándolo por 1. Esta operación da el valor de Factorial(2), por lo que la función vuelve a su primera ejecución extrayendo el siguiente valor 3 de la pila y multiplicándolo por el factorial, dando el valor 6, que finalmente devuelve la función

Recursión | uso de la pila en la recursión

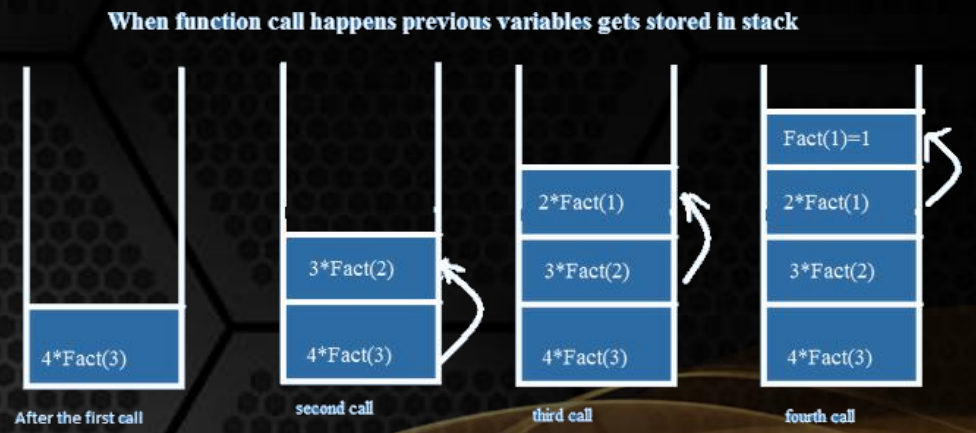
A partir del ejemplo anterior, observamos lo siguiente:

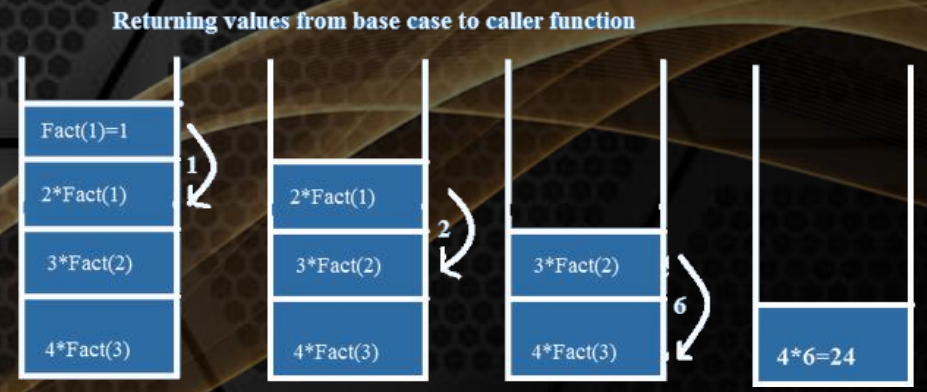
1. La función Factorial() en el Código de programa 4.2 se ejecuta tres veces para n = 3, de las cuales se llama a sí misma dos veces. El número de veces que una función se llama a sí misma se conoce como la profundidad recursiva de esa función.

2. Cada vez que la función se llama a sí misma, almacena una o más variables en la pila. Dado que las pilas contienen una cantidad limitada de memoria, las funciones con una alta profundidad recursiva pueden bloquearse debido a la falta de disponibilidad de memoria. Tal situación se conoce como desbordamiento de pila StackOverFlow.

3. Las funciones recursivas generalmente tienen (y de hecho deberían tener) una condición de terminación (o fin). La función Factorial() en el código de programa 4.2 deja de llamarse a sí misma cuando n = 1. Si esta condición no estuviera presente, la función seguiría llamándose a sí misma con los valores 3, 2, 1, 0, -1, -2, etc. Tal recursión se conoce como recursión sin fin.

4. Todas las funciones recursivas pasan por dos fases distintas. La primera fase, el bobinado, ocurre cuando la función se llama a sí misma y empuja los valores a la pila. La segunda fase, el desenrollamiento, se produce cuando la función saca valores de la pila, generalmente después de la condición final.






```
int factorial(int n) {

    if (int I <= 1){
        return 1;
    }
    else{
        return n * Factorial(n - 1);
    }
}
```



Recursión | **Ordenar una pila mediante recursión**



Dada una pila, ordenarla usando recursión. Uso de cualquier construcción de bucle como while, for.. etc no está permitido.
Solo podemos usar las siguientes funciones ADT en Stack S:

is_empty(S) : Tests whether stack is empty or not.
push(S) : Adds new element to the stack.
pop(S) : Removes top element from the stack.
top(S) : Returns value of the top element.
Note that this function does not remove element from the stack.

Ejemplo:

Input: -3 <--- Top
14
18
-5
30

Output: 30 <--- Top
18
14
-3
-5

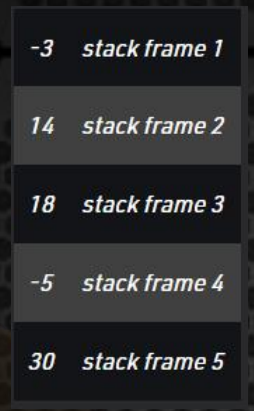
ESTE PROBLEMA ES PRINCIPALMENTE UNA VARIANTE DE LA PILA INVERSA QUE UTILIZA LA RECURSI N.
LA IDEA DE LA SOLUCI N ES MANTENER TODOS LOS VALORES EN FUNCTION CALL STACK HASTA QUE LA PILA SE VAC E. CUANDO LA PILA SE VAC E, INSERTE TODOS LOS ELEMENTOS RETENIDOS UNO POR UNO EN ORDEN ORDENADO. AQU EL ORDEN ORDENADO ES IMPORTANTE.

ALGORITMO
PODEMOS USAR EL SIGUIENTE ALGORITMO PARA ORDENAR LOS ELEMENTOS DE LA PILA:

Recursión | **Ordenar una pila mediante recursión**

Algoritmo
Podemos usar el siguiente algoritmo para ordenar los elementos de la pila:

```
sortStack(stack S)
    if stack is not empty:
        temp = pop(S);
        sortStack(S);
        sortedInsert(S, temp);
```



A continuación el algoritmo es para insertar el elemento está ordenado en orden:

```
sortedInsert(Stack S, element)
    if stack is empty OR element > top element
        push(S, elem)
    else
        temp = pop(S)
        sortedInsert(S, element)
        push(S, temp)
```

Now stack is empty so function insert in sorted order is called and it inserts 30 (from stack frame 5) at the bottom of the stack. Now stack looks like the below:

30

Now next element -5 (from stack frame 4) is picked. Since -5 < 30, -5 is inserted at the bottom of the stack. Now stack becomes:

30
-5

Next 18 (from stack frame 3) is picked. Since 18 < 30, 18 is inserted below 30. Now stack becomes:

30
18
-5

Next 14 (from stack frame 2) is picked. Since 14 < 30 and 14 < 18, it is inserted below 18. Now stack becomes:

30
18
14
-5

Now -3 (from stack frame 1) is picked, as -3 < 30 and -3 < 18 and -3 < 14, it is inserted below 14. Now stack becomes:

30
18
14
-3
-5

Ejemplo:

Input: -3 <--- Top	Output: 30 <--- Top
14	18
18	14
-5	-3
30	-5

Recursión | **Ordenar una pila mediante recursión**

```cpp
// C++ program to sort a stack using recursion
#include <iostream>
using namespace std;

// Stack is represented using linked list
struct stack {
    int data;
    struct stack* next;
};

// Utility function to initialize stack
void initStack(struct stack** s) { *s = NULL; }

// Utility function to check if stack is empty
int isEmpty(struct stack* s)
{
    if (s == NULL)
        return 1;
    return 0;
}

// Utility function to push an item to stack
void push(struct stack** s, int x)
{
    struct stack* p = (struct stack*)malloc(sizeof(*p));

    if (p == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return;
    }

    p->data = x;
    p->next = *s;
    *s = p;
}
```

```cpp
// Utility function to remove an item from stack
int pop(struct stack** s)
{
    int x;
    struct stack* temp;

    x = (*s)->data;
    temp = *s;
    (*s) = (*s)->next;
    free(temp);

    return x;
}

// Function to find top item
int top(struct stack* s) { return (s->data); }

// Recursive function to insert an item x in sorted way
void sortedInsert(struct stack** s, int x)
{
    // Base case: Either stack is empty or newly inserted
    // item is greater than top (more than all existing)
    if (isEmpty(*s) or x > top(*s)) {
        push(s, x);
        return;
    }

    // If top is greater, remove the top item and recur
    int temp = pop(s);
    sortedInsert(s, x);

    // Put back the top item removed earlier
    push(s, temp);
}
```

```cpp
// Function to sort stack
void sortStack(struct stack** s)
{
    // If stack is not empty
    if (!isEmpty(*s)) {
        // Remove the top item
        int x = pop(s);

        // Sort remaining stack
        sortStack(s);

        // Push the top item back in sorted stack
        sortedInsert(s, x);
    }
}
// Utility function to print contents of stack
void printStack(struct stack* s)
{
    while (s) {
        cout << s->data << " ";
        s = s->next;
    }
    cout << "\n";
}
// Driver code
int main(void)
{
    struct stack* top;
    initStack(&top);    push(&top, 30);
    push(&top, -5);    push(&top, 18);
    push(&top, 14);    push(&top, -3);

    cout << "Stack elements before sorting:\n";
    printStack(top);

    sortStack(&top);
    cout << "\n";

    cout << "Stack elements
            after sorting:\n";
    printStack(top);

    return 0;
}
```

[How to Sort a Stack using Recursion - GeeksforGeeks]

TALK IS CHEAP SHOW ME THE CODE

Stack.cpp



Tipos de Recursión| Recursion Types

Recursión

Introducción.

Las funciones son la característica más básica y útil de cualquier lenguaje de programación. Un conjunto de instrucciones que realiza operaciones lógicas, que pueden ser muy complejas y numerosos en número, se pueden agrupar como funciones (también llamados procedimientos).
Las funciones pueden llamarse a sí mismas o a otras funciones, y las funciones llamadas a su vez pueden llamar a la función de llamada. Este proceso se llama recursión y tales funciones se llaman
funciones recursivas. Una función recursiva hace que el programa sea compacto y legible.

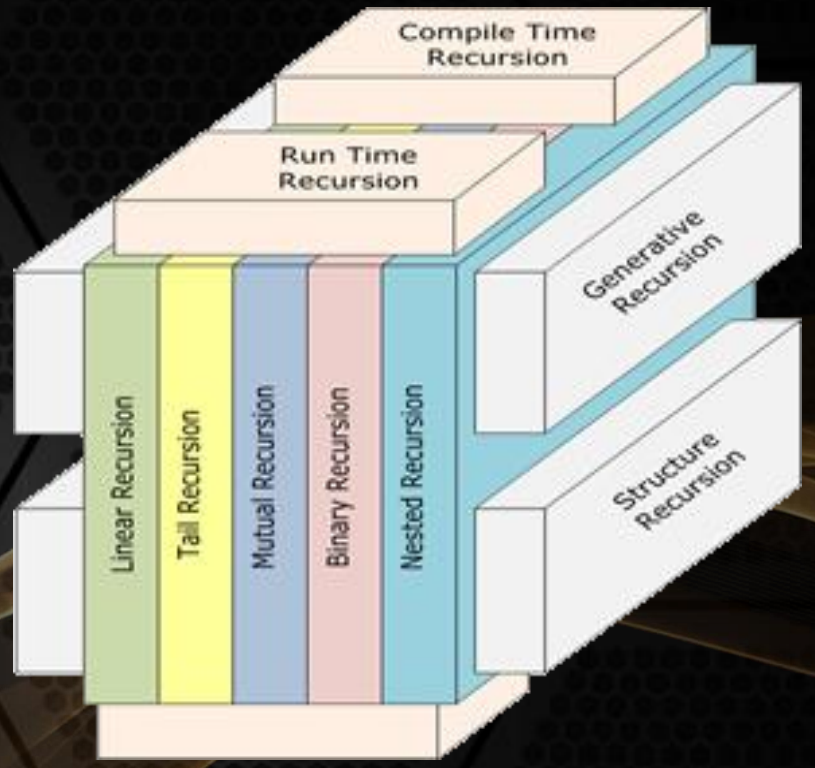
Recursión | Typess of Recursion C++

Tipos de recursión

En C++, los tipos de recursividad se pueden definir en más de una dimensión. En una dimensión, se puede clasificar como recursividad en tiempo de ejecución y recursión de tiempo de compilación utilizando metaprogramación de plantillas.

La recursión en **tiempo de ejecución** es la técnica de recursión más común utilizada en C++. Esto se puede implementar **cuando una función de C++ (o función miembro) se llama a sí misma**.

En C++, también podemos hacer recursividad de **tiempo de compilación** con la ayuda de la **metaprogramación de plantillas**. Al crear una instancia de una clase (o estructura) de plantilla en C++, el compilador creará el código de esa clase en tiempo de compilación. Al igual que la recursión en tiempo de ejecución, podemos crear una instancia de la propia clase de plantilla para realizar la recursión. Al igual que la recursión en tiempo de ejecución, también necesitamos la condición de terminación; de lo contrario, la instanciación irá para siempre, al menos teóricamente, pero, por supuesto, limitada a los recursos de la computadora y el compilador. En la metaprogramación de plantillas, podemos especificar la condición de terminación (o condición base) con la ayuda de la especialización de la plantilla o la especialización parcial de la plantilla, dependiendo de la condición de terminación.



https://www.codeproject.com/Articles/25470/Recursion-Primer-Using-C-Part-1

Se puede pensar que podríamos hacer lo mismo con el preprocesador de C++, usando macros, porque también serán reemplazadas durante la compilación. De hecho, técnicamente, el preprocesador reemplaza todas las macros incluso antes de la compilación, por lo que no está funcionando en tiempo de compilación. El preprocesador también tiene muchas limitaciones, como que no hay un símbolo de depuración definido para el depurador debido al simple reemplazo de texto, pero la limitación más crítica es que no puede ser recursivo.



La sección 16.3.4.2 del Estándar de C++ restringe estrictamente la escritura de macros que se llaman a sí mismas recursivamente, por lo tanto, no podemos hacer programación recursiva usando macros como lo hacemos en las plantillas.

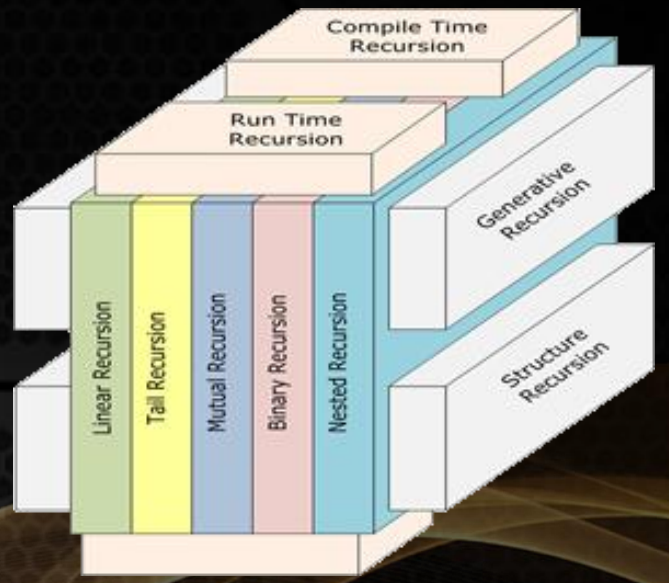
Recursión | Variants of Recursion

La recursión puede tener cualquiera de las siguientes formas:

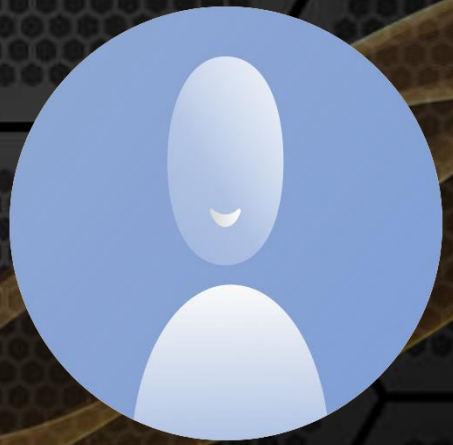
1. Una función se llama a sí misma.
2. Una función llama a otra función que a su vez llama a la función de llamada.
3. La llamada a la función es parte de la misma instrucción de procesamiento que realiza una llamada a la función recursiva.

Dependiendo de la siguiente caracterización, las funciones recursivas se clasifican como recursiones:

- Directas
- Indirectas
- Lineales
- Arbol
- Cola.



https://www.codeproject.com/Articles/25470/Recursion-Primer-Using-C-Part-1



Algunos términos más que se utilizan con respecto a la recursión se explican en la siguiente sección.

Recursión binaria Una función recursiva binaria se llama a sí misma dos veces. El cálculo de números de Fibonacci, la ordenación rápida y la ordenación de combinación son ejemplos de recursión binaria

El código de programa es un ejemplo de una recursión binaria como la función Fib() se llama a sí misma dos veces.

```
int Fib(n)
{
 if(n == 1 ||n == 2)
 return 1;
 else
 return(Fib(n - 1) + Fib(n - 2));
}
```

Recursión n-ary y permutaciones La forma más general de recursión es la recursión n-ary, donde n no es una constante sino algún parámetro de una función. Las funciones de este tipo son útiles para generar objetos combinatorios como las permutaciones.

Recursión | Variants of Recursion | Direct recursion

La recursión es cuando una función se llama a sí misma. Se dice que la recursión es directa cuando una función se llama a sí misma directamente, y se dice que es indirecta cuando llama a otra función que a su vez la llama. La función Factorial() que discutimos es un ejemplo de recursión directa.

```
int Power(int x, int y)
{
 if(y == 1)
 return x;
 else
 return (x * Power(x, y - 1));
}
```

Recursión | Variants of Recursion | indirect recursion



Se dice que una función es indirectamente recursiva si llama a otra función, que a su vez la llama. El código de programa es un ejemplo de recursión indirecta, donde la función Fact() llama a la función Dummy(), y la función Dummy() a su vez llama Fact().

```
int Fact(int n)
{
 if(n <= 1)
 return 1;
 else
 return (n * Dummy(n - 1));
}
void Dummy(int n)
{
 Fact(n);
}
```

Recursión | Variants of Recursion | tail recursion

Se dice que una función recursiva es recursiva de cola si no hay operaciones pendientes que realizar a la vuelta de una llamada recursiva.

La recursión de cola también se utiliza para devolver el valor de la última llamada recursiva como el valor de la función. La recursividad de cola es ventajosa ya que la cantidad de información que debe almacenarse durante el cálculo es independiente del número de llamadas recursivas. La función Factorial() en el Código de Programa es un ejemplo de una función recursiva no tail. La función Binary_Search() en el código de programa es un ejemplo de una función recursiva de cola.

```
int Binary_Search(int A[], int low, int high, int key)
{
      int mid;
      if (low <= high)

    {
          mid = (low + high) / 2;
          if (A[mid] == key)
 return mid;
          else  if (key < A[mid])
 return Binary_Search(A, low, mid - 1, key);
          else  return Binary_Search(A, mid + 1, high, key);

    }
     return - 1;
}
```



Recursión | Variants of Recursion | Linear Recursion

Dependiendo de la forma en que crece la recursión, se clasifica como lineal o arbórea. Se dice que una función recursiva es linealmente recursiva cuando ninguna operación pendiente implica otra llamada recursiva, por ejemplo, la función Fact(). Esta es la forma más simple de recursión y ocurre cuando una acción tiene una estructura repetitiva simple que consiste en algunos pasos básicos seguidos por la acción nuevamente. La función Factorial() en el Código de Programa 4.2 es un ejemplo de recursión lineal.

```
//Program Code 4.2
int Factorial(int n)
{
 if(n == 1) // end condition
 return 1;
 els e
 return Factorial(n - 1) * n;
}
```



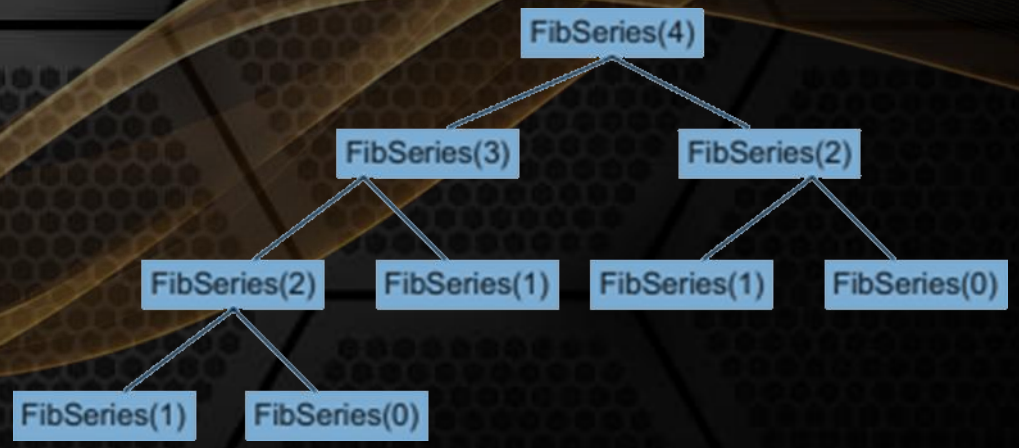
Recursión | Variants of Recursion | Tree Recursion



En una función recursiva, si hay otra llamada recursiva en el conjunto de operaciones que se completarán después de que termine la recursión, esto se denomina recursión de árbol. Ejemplos de funciones recursivas de árbol son los algoritmos de ordenación rápida y de combinación, el algoritmo FibSeries, etc. La función de Fibonacci FibSeries() se define como

FibSeries(n) = 0, if n = 0
= 1, if n = 1
= FibSeries(n - 1) + FibSeries(n - 2), otherwise Let n = 5.

FibSeries(0) = 0
FibSeries(1) = 1
FibSeries(2) = FibSeries(0) + FibSeries(1) = 1
FibSeries(3) = FibSeries(1) + FibSeries(2) = 2
FibSeries(4) = FibSeries(2) + FibSeries(3) = 3
FibSeries(5) = FibSeries(3) + FibSeries(4) = 5

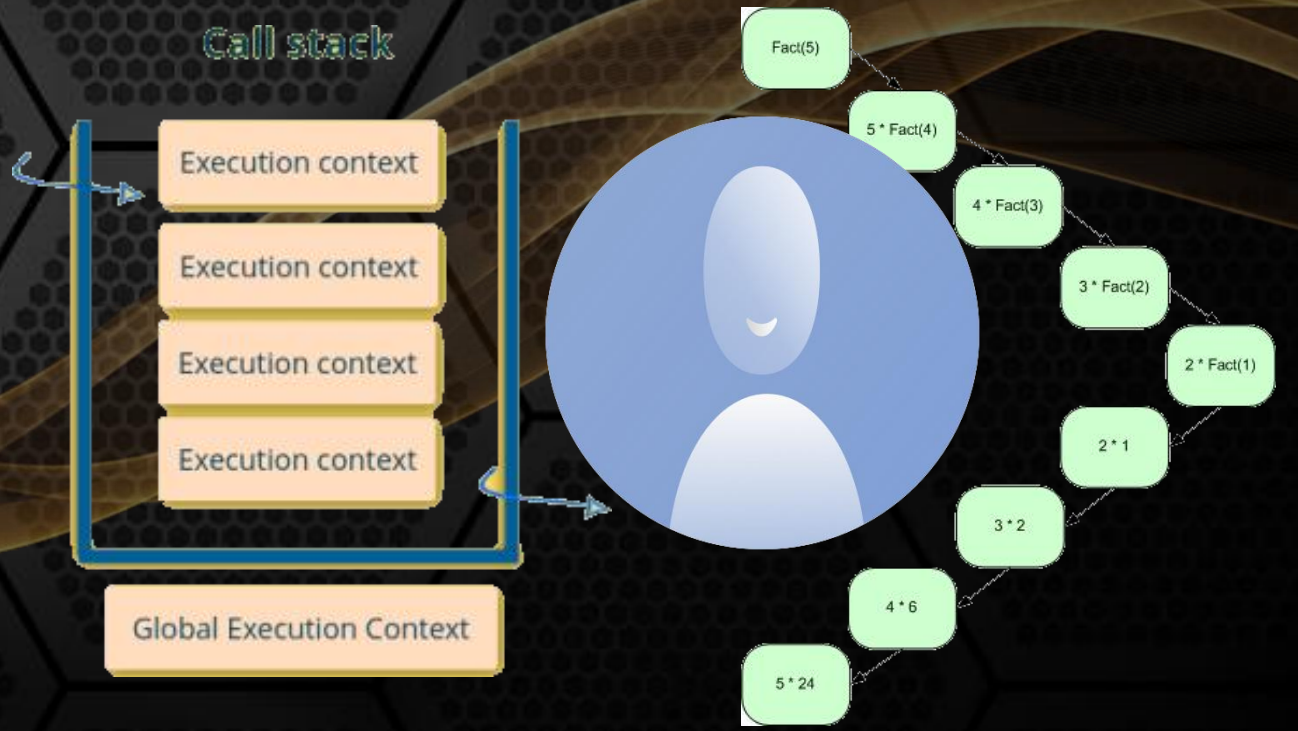


Recursión | Execution of recursive calls

Veamos ahora cómo se ejecutan las llamadas recursivas. En cada llamada recursiva, todos los parámetros de referencia y variables locales se colocan en la pila junto con el valor de la función y la dirección de retorno. Los datos se colocan conceptualmente en un marco de pila, que se empuja a la pila del sistema.
Un marco de pila contiene cuatro elementos diferentes:

1. Los parámetros de referencia que debe procesar la función llamada
2. Variables locales en la función de llamada
3. La dirección de devolución
4. La expresión que va a recibir el valor devuelto, si lo hubiera.

```
//Program Code 4.2
int Factorial(int n)
{
 if(n == 1) // end condition
 return 1;
 els e
 return n * Factorial(n - 1) ;
}
```



Recursión | recursive Functions



La recursión suele ser vista por los estudiantes como una técnica mística que es útil solo para alguna clase muy especial de problemas, como los factoriales de computación o la serie de Fibonacci. Esto no es cierto. *Prácticamente, cualquier función escrita usando un código iterativo se puede convertir en un código recursivo.* Por supuesto, esto no garantiza que el programa resultante sea fácil de entender, pero a menudo, el programa da como resultado un código compacto y legible. Veamos cuándo la recursión es una solución adecuada. Un caso es cuando el problema en sí se define recursivamente. Ejemplos apropiados de esto podrían ser los coeficientes factoriales y binomiales.

1. $n! = n \times (n-1)!\ \{\text{if } n = 1, n! = 1\}$
2. $\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$
3. $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$
4. $x^y = x \times x^{y-1}$

Las funciones Recursivas son a menudo simples y elegantes, y su corrección se puede verificar fácilmente. Muchas funciones matemáticas se definen recursivamente, y su traducción a un lenguaje de programación es a menudo fácil. La recursión es natural en Ada, Algol, C, C++, Haskell, Java, Lisp, ML, Modula, Pascal y muchos otros lenguajes de programación.

Cuando se usa descuidadamente, la recursión a veces puede resultar en una función ineficiente. Las funciones recursivas están estrechamente relacionadas con las definiciones inductivas de funciones en matemáticas.

Recursión | Writing Recursive Code



El enfoque general para escribir una función recursiva se enumera en la siguiente secuencia:

1. Escriba el encabezado de la función para que esté seguro de lo que hará la función y cómo se llamará. Identifique alguna unidad de medida para el tamaño del problema en el que trabajará la función o el procedimiento. Luego, finja que la tarea es escribir una función que funcione en problemas de todos los tamaños.

2. Descomponer el problema en subprocesos. Identificar claramente el caso no recursivo del problema. Hazlo lo más pequeño posible. La función casi siempre comenzará probando este caso no recursivo, también conocido como el caso base o la condición final.

3. Escribir llamadas recursivas para resolver aquellos subprocesos cuya forma sea similar a la del problema original.

4. Escriba el código para combinar, mejorar o modificar los resultados de la(s) llamada(s) recursiva(s), si es necesario, para construir el valor de retorno deseado o crear los efectos secundarios deseados.

5. Escriba la(s) condición(es) final(es) para manejar cualquier situación que no sea manejada correctamente por la parte recursiva del programa.

Recursión | Torre de Hanoi: un ejemplo de recursión

Programa para la Torre de Hanoi



La Torre de Hanoi es un rompecabezas matemático donde tenemos tres varillas y n discos. El objetivo del rompecabezas es mover toda la pila a otra varilla, obedeciendo las siguientes reglas simples:

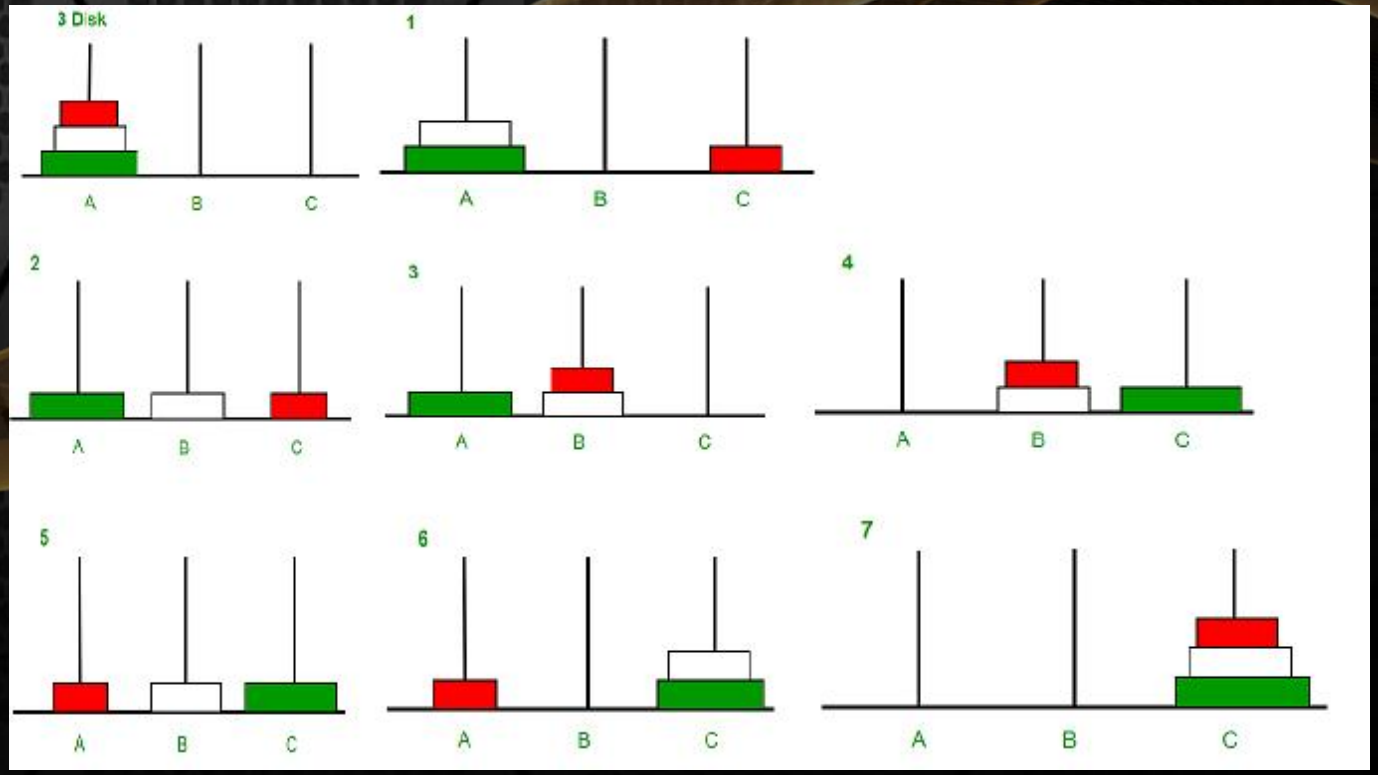
1.Solo se puede mover un disco a la vez.

2.Cada movimiento consiste en tomar el disco superior de una de las pilas y colocarlo encima de otra pila, es decir, un disco solo se puede mover si es el disco superior de una pila.

3.No se puede colocar ningún disco encima de un disco más pequeño.

Take an example for 2 disks :
Let rod 1 = 'A', rod 2 = 'B', rod 3 = 'C'.
Step 1 : Shift first disk from 'A' to 'B'.
Step 2 : Shift second disk from 'A' to 'C'.
Step 3 : Shift first disk from 'B' to 'C'.

The pattern here is :
Shift 'n-1' disks from 'A' to 'B'.
Shift last disk from 'A' to 'C'.
Shift 'n-1' disks from 'B' to 'C'.
Image illustration for 3 disks :

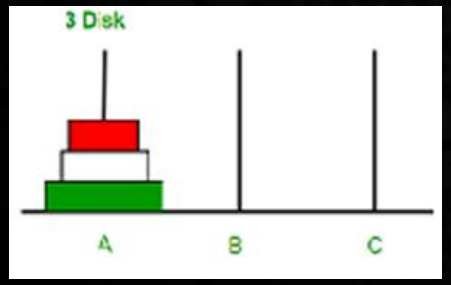


Recursión | Torre de Hanoi

Programa para la Torre de Hanoi

Input : 2
Output :
Disk 1 moved from A to B
Disk 2 moved from A to C
Disk 1 moved from B to C

Input : 3
Output :
Disk 1 moved from A to C
Disk 2 moved from A to B
Disk 1 moved from C to B
Disk 3 moved from A to C
Disk 1 moved from B to A
Disk 2 moved from B to C
Disk 1 moved from A to C




```cpp
// C++ recursive function to
// solve tower of hanoi puzzle
#include <iostream>
using namespace std;

void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod)
{
    if (n == 1)
    {
        cout << "Move disk 1 from rod " << from_rod <<  " to rod " << to_rod<<endl;
        return;
    }
    towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);
    cout << "Move disk " << n << " from rod " << from_rod << " to rod " << to_rod << endl;
    towerOfHanoi(n - 1, aux_rod, to_rod, from_rod);
}

// Driver code
int main()
{
    int n = 4; // Number of disks
    towerOfHanoi(n, 'A', 'C', 'B'); // A, B and C are names of rods
    return 0;
}
```

```
Move disk 1 from rod A to rod B
Move disk 2 from rod A to rod C
Move disk 1 from rod B to rod C
Move disk 3 from rod A to rod B
Move disk 1 from rod C to rod A
Move disk 2 from rod C to rod B
Move disk 1 from rod A to rod B
Move disk 4 from rod A to rod C
Move disk 1 from rod B to rod C
Move disk 2 from rod B to rod A
Move disk 1 from rod C to rod A
Move disk 3 from rod B to rod C
Move disk 1 from rod A to rod B
Move disk 2 from rod A to rod C
Move disk 1 from rod B to rod C
```





Recursión | Writing Recursive Code - Example

En el algoritmo 4.4, en la función de llamador, se realiza una llamada a HTower con disco = 5, fuente = A, dest = B y repuesto = C.



ALGORITHM 4.4

```
HTower(disk, source, dest, spare)
if disk == 0, then
   move disk from source to dest
else
   HTower(disk - 1, source, spare, dest)              // Step 1
   move disk from source to dest                  // Step 2
   HTower(disk - 1, spare, dest, source)              // Step 3
end if
```

Tenga en cuenta que el pseudocódigo agrega un caso base cuando disco = 0, es decir, el disco más pequeño. En este caso, no necesitamos preocuparnos por discos más pequeños, por lo que podemos simplemente mover el disco directamente. En los otros casos, seguimos la función recursiva de tres pasos ya descrita para el disco 5. La representación arbórea de las llamadas recursivas se muestra en la Fig. 4.6.

La raíz representa la primera llamada a la función. La llamada a la función se representa como un nodo en el árbol. Los nodos secundarios del nodo n representan las llamadas a funciones realizadas por n. Por ejemplo, HTower(2, A, C, B) y HTower(2, C, B, A) son los nodos hijos de HTower(3, A, B, C) ya que estas son las dos llamadas a funciones que HTower(3, A, B, C) realiza. Los nodos hoja representan los casos base.

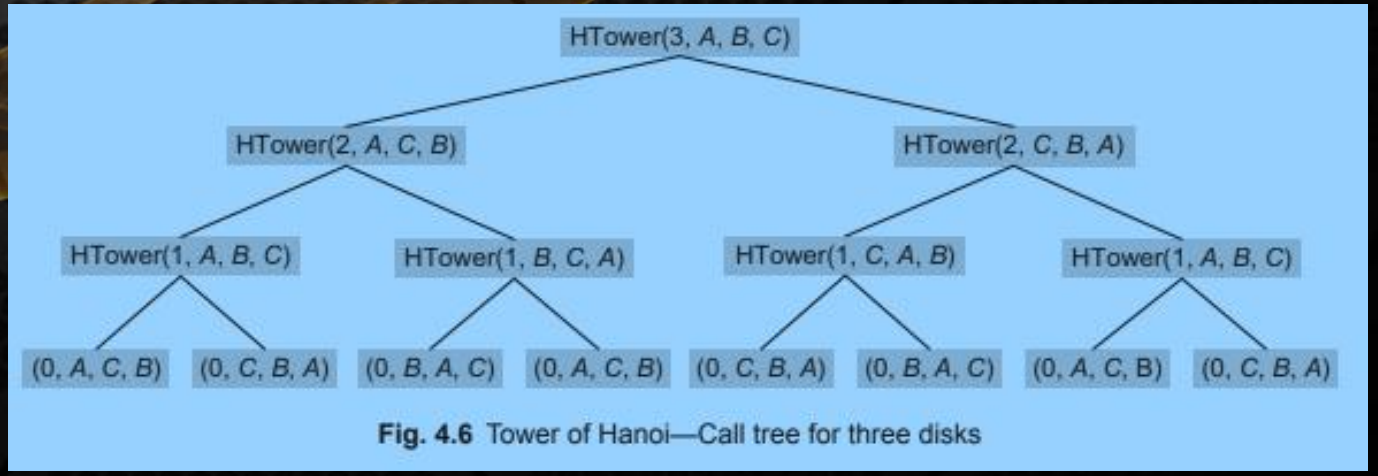


Fig. 4.6 Tower of Hanoi—Call tree for three disks

Recursión | Comprobación de la corrección

Uno de los aspectos más difíciles de programar recursivamente es el proceso de aceptar que la llamada recursiva hará lo correcto. La siguiente lista de comprobación proporciona las cinco condiciones que deben cumplirse para que la recursión funcione. Si cada una de estas se mantiene para una función recursiva, se puede concluir que la recursión funcionará correctamente.

1. Una función recursiva debe tener al menos una condición final y un caso recursivo.
2. La prueba para la condición final debe ejecutarse antes de la llamada recursiva.
3. El problema debe desglosarse de tal manera que la llamada recursiva esté más cerca del caso base que la llamada de nivel superior. Esta condición en realidad no es del todo fuerte o suficiente. Avanzar hacia la condición final por sí solo no es suficiente; también debe ser cierto que el caso base se alcanza en un número finito de llamadas recursivas.
4. La llamada recursiva no debe omitir el caso base.
5. Verifique que el código no recursivo de la función esté funcionando correctamente.



Los siguientes puntos deben tenerse en cuenta al hacer programación recursiva:

1. Las funciones recursivas se llaman a sí mismas dentro de su propia definición.
2. Las funciones recursivas deben tener una condición de terminación no recursiva; de lo contrario, se producirá un bucle infinito.
3. La recursión, aunque fácil de codificar, a menudo, pero no siempre, priva de memoria

Recursión | Iteración versus Recursión

La recursión es un enfoque de arriba hacia abajo de la resolución de problemas. Divide el problema en pedazos o selecciona un paso clave, posponiendo el resto. Por otro lado, la iteración es más un enfoque de abajo hacia arriba. Comienza con lo que se conoce y a partir de esto construye la solución paso a paso. Es difícil decir que la versión no recursiva es mejor que la recursiva o viceversa. Sin embargo, algunos lenguajes no admiten la escritura de código recursivo, como FORTRAN o COBOL. La versión no recursiva es más eficiente ya que la sobrecarga del paso de parámetros en la mayoría de los compiladores es mucho mas costoso.

Deméritos de algoritmos recursivos Aunque con muchos méritos, los algoritmos recursivos tienen sus limitaciones. Son los siguientes:

1. Muchos lenguajes de programación no soportan recursión; por lo tanto, la función matemática recursiva debe implementarse utilizando métodos iterativos.

2. Aunque las funciones matemáticas se pueden implementar fácilmente utilizando la recursión, siempre es a costa de tiempo de ejecución adicional y espacio de memoria. Por ejemplo, tomemos el caso de un árbol de recursión para generar seis números en una serie de Fibonacci. Se sabe que una serie de Fibonacci es de la forma 0, 1, 1, 2, 3, 5, 8, 13, ..., n, donde cada número del tercero es la suma de los dos números anteriores. Se puede notar que F(n - 2) se calcula dos veces, F(n - 3) se calcula tres veces y F(n - 4) se calcula cuatro veces.

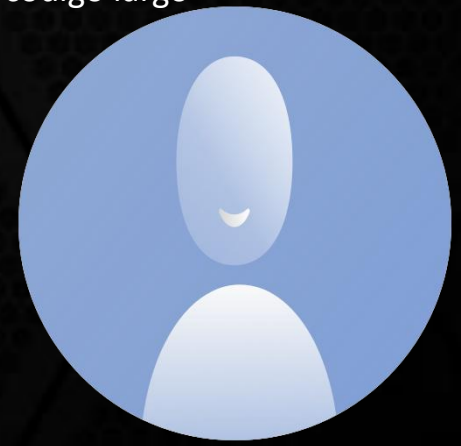
3. Se puede llamar a una función recursiva desde dentro o fuera de sí misma, y para garantizar un funcionamiento adecuado, debe guardar las direcciones de retorno en algún orden para que el retorno a la ubicación adecuada produzca el resultado deseado cuando se realice el retorno a una instrucción de llamada.

Deméritos de los métodos iterativos Aunque el método iterativo tiene varios méritos, también tiene sus propias limitaciones. Son los siguientes:

1. El código iterativo no es legible y, por lo tanto, no es fácil de entender.

2. En las técnicas iterativas, el bucle de las declaraciones es necesario y necesita una lógica compleja.

3. Las iteraciones pueden dar lugar a un código largo



Recursión | Simulación de recursión mediante stack (Eliminación de la recursión)

Dondequiera que un objeto/proceso/relación de datos se defina recursivamente, a menudo es fácil describir los algoritmos de forma recursiva. Si un lenguaje de programación no admite la recursividad o uno necesita un código no recursivo, entonces un código recursivo se puede traducir a uno no recursivo. Una vez que se escribe una función recursiva y se verifica su corrección, se puede eliminar la recursividad para mayor eficiencia. Esto se puede hacer usando las siguientes reglas:

1. Al comienzo de la función recursiva, se inserta un código para crear una pila vacía. Esta pila se debe utilizar para contener los valores de los parámetros, las variables locales, el valor de la función y la dirección de retorno para cada llamada recursiva.

2. La etiqueta de salto se adjunta a la primera instrucción ejecutable, digamos label_1. Ahora, reemplace cada llamada recursiva por un conjunto de instrucciones que realicen lo siguiente:

    1. Empuje los valores de todos los parámetros y variables locales en la pila.
    2. Crear la nueva etiqueta, label_i y almacenar i en la pila. El valor i de esta etiqueta se utilizará para calcular la dirección de retorno. Esta etiqueta se coloca en el programa como se describe en el paso 2(e).
    3. Evaluar los argumentos de esta convocatoria, que pueden ser parte de la expresión. Asigne estos valores a los parámetros formales apropiados.
    4. Inserte una rama incondicional al principio de la función. e) Adjunte la etiqueta creada en el paso 2 b) a la declaración inmediatamente después de la rama incondicional.
    5. Adjunte la etiqueta a una instrucción que recupere el valor de la función de la parte superior de la pila. Luego, haga uso de este valor de cualquier manera que el programa recursivo describa.

Recursión | Simulación de recursión mediante stack (Eliminación de la recursión)

3. Una vez que se hayan eliminado todas las llamadas recursivas, reemplace todas las instrucciones return siguiendo los siguientes pasos:

a) Si la pila está vacía, ejecute un retorno normal.
b) b) De lo contrario, tome el valor actual de todos los parámetros de salida (explícita o implícitamente entendidos como de tipo salida o entrada) y asigne estos valores a las variables correspondientes que se encuentran en la parte superior de la pila.
c) c) Ahora, inserte un código que elimine el índice de la dirección de retorno de la pila si se ha colocado alguno allí. Asigne esta dirección a alguna variable no utilizada.
d) d) Eliminar los valores de todas las variables y parámetros locales de la pila y asignarlos a sus variables correspondientes.
e) e) Si se trata de una función, inserte instrucciones para evaluar la expresión inmediatamente después de return() y almacene el resultado en la parte superior de la pila.
f) Utilizar el índice de la etiqueta de la dirección de retorno para ejecutar una bifurcación en esa etiqueta. Si todas estas reglas se siguen cuidadosamente, se puede convertir la recursión en un código iterativo.

C++ admite la recursión y se controla mediante una pila en tiempo de ejecución. Para cada llamada a la función, todos los parámetros reales se colocan en la pila. Esto también se denomina registro de activación. Este registro de activación contiene memoria para el valor devuelto: un puntero a la base del fotograma de pila anterior de la pila. Incluye la dirección de retorno, es decir, la dirección de la instrucción que se ejecutará después de que se complete la llamada a la función. También incluye memoria para todos los parámetros y para todas las variables locales de la función. El funcionamiento de la recursión es como se describió anteriormente

Recursión | Aplicaciones - Resumen

Aplicaciones de recursión Las siguientes son las principales áreas donde se puede aplicar el proceso de recursión:

1. Inteligencia artificial
2. Técnicas de búsqueda
3. Juego
4. Lingüística computacional y procesamiento del lenguaje natural
5. Sistemas expertos
6. Reconocimiento de patrones y visión artificial
7. Robótica



Conclusiones:

•Una función puede llamarse a sí misma u otras funciones, y las funciones llamadas a su vez pueden volver a llamar a la función de llamada. Tales funciones se llaman funciones recursivas.
• Cualquier código iterativo correcto se puede convertir en su código recursivo equivalente y viceversa.
• Los conceptos básicos y las ideas involucradas con la recursión son simples: una función que debe resolverse se trata como un gran problema y se resuelve a sí misma al usarse a sí misma para resolver un problema un poco más pequeño. La relación de recurrencia se convierte fácilmente en código recursivo.
• El funcionamiento de la recursión es bastante sencillo. Sin embargo, para comprender mejor el funcionamiento de la recursión y poder usarla bien, se requiere práctica. La mejor manera de obtener esto es escribir muchas funciones recursivas.
• La recursión se puede utilizar para dividir y conquistar algoritmos de búsqueda y ordenación para aumentar la eficacia de estas operaciones.
• Para la mayoría de los problemas, como la Torre de Hanoi, la recursión presenta una solución increíblemente elegante que es fácil de codificar y fácil de entender.

Recursión | Resumen

Recursividad binaria Una función recursiva unaria simple se llama a sí misma una vez, mientras que la función recursiva binaria se llama a sí misma dos veces. Un factorial es una función unaria, mientras que Fibonacci es una recursión binaria.

Profundidad de recursión El número de veces que una función se llama a sí misma se conoce como la profundidad recursiva de esa función.

Recursión directa e indirecta Cuando una función recursiva se llama a sí misma directamente, se llama recursión directa y cuando la función llama a otra función, que a su vez llama a la primera función, se llama recursión indirecta.

Condición final Las funciones recursivas generalmente tienen y de hecho deberían tener una condición que terminaría las llamadas recursivas. Esta condición de terminación se denomina condición de finalización. En la función factorial, cuando n = 1 la función devuelve 1. Si esta condición no estuviera presente, la función seguiría llamándose a sí misma con los valores 3, 2, 1, 0, -1, -2, y así sucesivamente hasta la infi nidad. Tal recursión se conoce como recursión sin fin.

Recursión lineal y arbórea Dependiendo de la forma en que crece la recursión, se clasifica como lineal o arbórea. Se dice que una función recursiva es linealmente recursiva cuando ninguna operación pendiente implica otra llamada recursiva. Si hay otra llamada recursiva en el conjunto de operaciones que se completarán después de que termine la recursión, entonces se denomina recursión de árbol. Factorial es un ejemplo de recursión lineal y Fibonacci es un ejemplo de recursión de árboles.

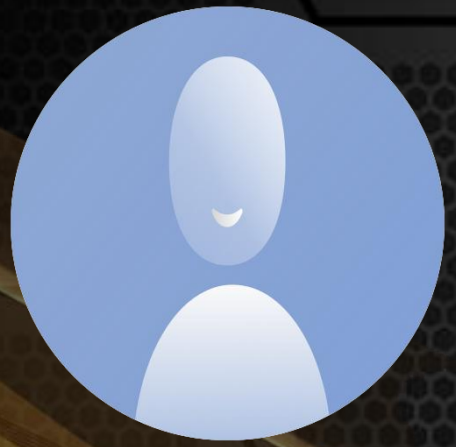
Relación de recurrencia Una recurrencia es una función matemática bien demostrada escrita en términos de sí misma; es una función matemática desafiada recursivamente como n! = n × (n - 1)!

Funciones recursivas Una función puede llamarse a sí misma o llamar a otras funciones y las funciones llamadas a su vez de nuevo pueden llamar a la función de llamada. Tales funciones se llaman funciones recursivas.

Stack overfl ow en recursión Cada vez que una función se llama a sí misma, almacena una o más variables en la pila. Dado que la pila contiene una cantidad limitada de memoria, las funciones con una alta profundidad recursiva pueden bloquearse debido a la falta de disponibilidad de memoria. Tal situación se conoce como stack overfl ow.

Recursividad de cola Se dice que una función recursiva es recursiva de cola si no hay operaciones pendientes que realizar a la vuelta de una llamada recursiva; de lo contrario, se denomina no recursión de cola. La función factorial es un ejemplo de recursión no cola, mientras que la búsqueda binaria es un ejemplo de recursión cola.

Bobinado y desenrollamiento de la recursividad Todas las funciones recursivas pasan por dos fases distintas. La primera fase, el bobinado, ocurre cuando la función se llama a sí misma y empuja valores a la pila. La segunda fase, el desenrollamiento, se produce cuando la función está haciendo estallar valores de la pila, generalmente después de la condición final.



Ejercitar los LABS



Muchas Gracias.